AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

408/1, Kuratoli, Khilkhet, Dhaka 1229, Bangladesh

Mid-Term Project Report

Course Title: Programming in Python
Course Code: CSC 4262 | Section: A
Semester: 2025-2026, Spring

"Student Budget Tracker"

Submitted To:
MD. Tanzeem Rahat
Faculty, Department of Computer Science

Submitted By:
Durjoy Roy
Student ID: 23-50576-1
Section: A
Group: 17

Date of Submission: March 28, 2026

1. Project Overview

For this mid-term project, I chose the CLI Project track and built a simple budget tracking application called Student Budget Tracker. The idea came from my own experience of losing track of daily expenses, things like rickshaw fares, photocopy costs, and mobile recharges add up quickly but are easy to forget.

I designed the app specifically for a Bangladeshi student's daily life. Instead of generic categories like 'rent' or 'groceries', I used categories that are actually relevant; rickshaw, photocopy, university fee, tuition, mobile recharge, and so on. The program runs entirely in the terminal and saves all data to a JSON file so that records are not lost when the program is closed.

Track: **CLI Project (Track 3)** | External Library: **rich** | Data File: **budget_data.json**

2. Environment Setup

To run this project, follow these steps in order:

Step 1 : Create a virtual environment

```
python3 -m venv .venv
```

Step 2 : Activate the virtual environment (macOS / Linux)

```
source .venv/bin/activate
```

After running this, the terminal prompt will show (.venv) at the start, which means the environment is active.

Step 3 : Install required packages

```
pip install -r requirements.txt
```

Step 4 : Run the program

```
python3 main.py
```

Step 5 : Freeze dependencies (already done)

```
pip freeze > requirements.txt
```

The requirements.txt file is already included in the submission.

3. OOP Design

I used four classes in this project, each with a clear and separate responsibility. The classes are connected using composition ; BudgetCLI holds a BudgetManager object, and BudgetManager holds a Storage object. This way, each class only does its own job.

Class	Role	Key Methods
Entry	Entity class — holds data for one income or expense entry.	__init__, to_dict, from_dict
Storage	Handles reading and writing data to the JSON file.	save_data, load_data
BudgetManager	Contains all the main logic — add, edit, delete, search, sort, and report generation.	add_entry, update_entry, delete_entry, search_by_category, sort_entries, get_monthly_summary
BudgetCLI	Handles the menu, user input, and displaying results in the terminal.	run, show_menu, handle_add, handle_view, handle_edit, handle_delete, handle_search, handle_sort, handle_monthly_report

Each class has a constructor (__init__) and its own methods. The Entry class also has to_dict() and from_dict() methods for converting between objects and JSON format, which is needed for saving and loading data.

4. Features

The application has a while-loop based menu with 7 main options plus an exit. Below is a description of each feature:

4.1 Add New Entry

This lets the user record a new income or expense. The program asks for the type (income or expense), amount, category, an optional note, and date. If the user presses Enter on the date field, today's date is automatically used. All inputs are validated for example, entering a letter instead of a number for the amount will show an error and ask again instead of crashing.

4.2 View All Entries

Displays all saved entries in a clean table using the rich library. Income rows are shown in green with a + sign, and expense rows are shown in red with a - sign. The total number of entries is shown at the bottom.

4.3 Edit an Entry

The user can update the amount, category, or note of any existing entry by entering its ID. If the user just presses Enter on any field, the current value stays unchanged. This makes editing flexible without forcing the user to retype everything.

4.4 Delete an Entry

The user enters an ID to delete. Before deleting, the program asks for confirmation (y/n) to prevent accidental deletion. The user can also type 'back' to cancel.

4.5 Search by Category

The user can search entries by typing a category keyword. The search is case-insensitive, so 'Rickshaw' and 'rickshaw' both give the same result. If no entries match, a friendly message is shown instead of an empty screen.

4.6 Sort Entries

Entries can be sorted in four ways: by amount from high to low, low to high, by date from newest to oldest, or oldest to newest. The result is shown in a labeled table so the user knows which sorting was applied.

4.7 Monthly Summary Report

The user enters a year and month, and the program calculates total income, total expense, net balance, and the category where the most money was spent. If expenses are 80% or more of income, a budget warning message is shown. This was one of the features I was most interested in building.

5. Data Persistence

All data is saved to a file called budget_data.json using Python's built-in json module. The file is created automatically the first time an entry is added. Every time the program starts, the Storage class loads all existing entries from this file.

Each entry in the JSON file looks like this:

```
{
  "id": 1,
  "type": "expense",
  "amount": 11000,
  "category": "rent",
  "note": "",
  "date": "2026-03-28"
}
```

IDs are generated automatically using max(existing_ids) + 1, so they are always unique. If the JSON file gets corrupted somehow, the load_data() method catches the error and returns an empty list instead of crashing the program.

6. Validation and Error Handling

I tried to make sure the program does not crash on any wrong input. Here is what is handled:

- Wrong type input (anything other than income or expense) : re-asks until valid
- Non-numeric amount or zero/negative amount : catches ValueError, shows message, re-asks
- Empty category : not accepted, user must type something
- Wrong date format : validated using datetime.strptime, re-asks on failure
- Invalid or non-existent entry ID : caught in a while loop, does not kick user out
- Delete confirmation : requires 'y' before anything is deleted
- Empty search result : shows a message, does not crash
- Corrupted JSON file : caught with JSONDecodeError, returns empty list
- Division by zero in budget warning : checked with if total_income > 0 before dividing
- 'back' keyword : available in most input fields to cancel and return to the menu

7. Challenges Faced

The part I found most difficult was date validation. At first I only checked if the format looked correct (like splitting by '-' and checking for digits), but that still allowed invalid dates like 2026-13-99 to pass through. I then learned about datetime.strptime and used it to actually verify the date, which fixed the problem properly.

Another issue I ran into was with Python's indentation. While restructuring the BudgetCLI class and moving some functions around, the handle_sort and handle_monthly_report methods accidentally ended up outside the class due to an indentation error. The program was not crashing at startup, so it took me a bit to figure out why options 6 and 7 were not working. After checking the indentation block carefully, I found and fixed the issue.

I also had a small bug where handle_view() had an input() call at the bottom to pause the screen. This caused a problem when handle_edit() called handle_view() ,the user had to press Enter an extra time before the ID prompt appeared. Removing that line fixed the flow.

Overall, working on this project helped me understand OOP much better, especially how composition works in practice. Writing and debugging the code myself made the concepts feel more clear than just reading about them.

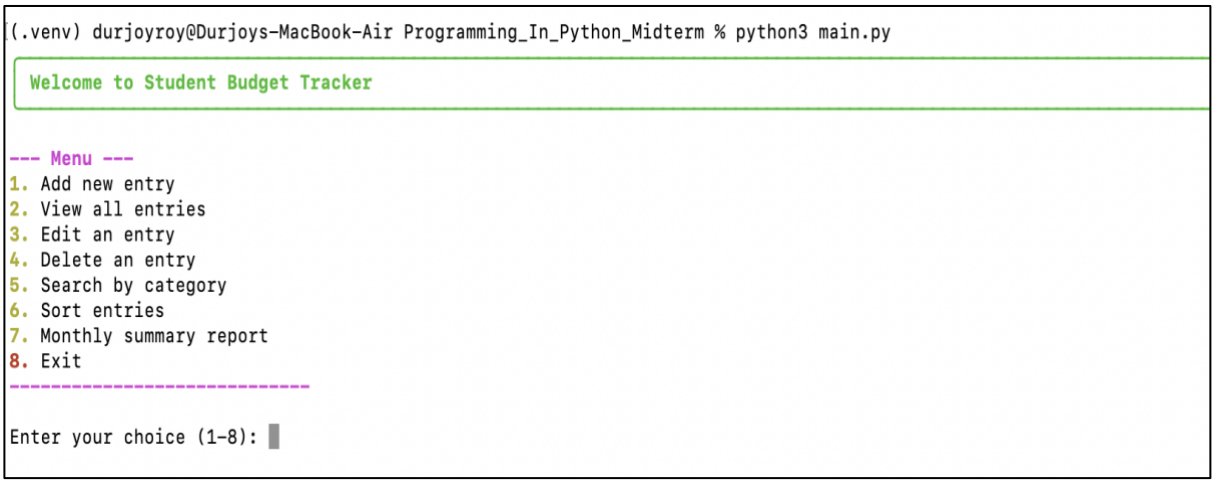
8. Marking Scheme Coverage

Category	Weight	How I covered it
Environment + packaging	10%	Created .venv, installed rich, generated requirements.txt, setup steps included in this report.
OOP design	25%	Used 4 classes with constructors and methods. Composition shown — BudgetCLI uses BudgetManager, which uses Storage.
Functionality	30%	Full CRUD operations, search by category, sort by amount and date, monthly summary report with warning.
Persistence + data integrity	15%	All data saved to budget_data.json using json module. Data loads on startup. IDs are auto-generated and stable.
Validation + edge cases	10%	Every input is validated. No crashes on wrong input. Friendly error messages. 'back' option to cancel anytime.
Clarity + demo	10%	Color-coded rich tables in terminal. Clear menu. Screenshots included in this report.

9. Demo Screenshots

The screenshots below show the main features of the program running in the terminal:

- Screenshot 1 : Welcome screen and main menu



- Screenshot 2 : Adding a new expense entry

```
--- Menu ---
1. Add new entry
2. View all entries
3. Edit an entry
4. Delete an entry
5. Search by category
6. Sort entries
7. Monthly summary report
8. Exit
-----------------------------

Enter your choice (1-8): 1

--- Add New Entry ---
Type (income/expense or 'back' to cancel): expense
Amount (৳): 2000
Hint: food, transport, photocopy, recharge, etc.
Category: food
Note (optional):
Date (YYYY-MM-DD, press Enter for today): 2026-03-20

 Entry added successfully! ID: 5
```

- Screenshot 3 : View all entries table with color-coded rows

```
--- Menu ---
1. Add new entry
2. View all entries
3. Edit an entry
4. Delete an entry
5. Search by category
6. Sort entries
7. Monthly summary report
8. Exit
-----------------------------

Enter your choice (1-8): 2

--- All Entries ---
```

ID	Type	Amount (৳)	Category	Note	Date
1	Income	+6000.00	Tuition	-	2026-03-28
2	Income	+1000.00	Gift	from friend	2026-03-28
3	Expense	-4500.00	Other	-	2026-03-28
4	Income	+10000.00	Allowance	-	2026-03-28
5	Expense	-2000.00	Food	-	2026-03-20

Total entries: 5

- Screenshot 4 : Search by category result

```
--- Menu ---
1. Add new entry
2. View all entries
3. Edit an entry
4. Delete an entry
5. Search by category
6. Sort entries
7. Monthly summary report
8. Exit
-----------------------------

Enter your choice (1-8): 5

--- Search by Category ---
Enter category to search (or press Enter to cancel): gift
```

ID	Type	Amount (৳)	Category	Note	Date
2	Income	+1000.00	Gift	from friend	2026-03-28

```
Found 1 result(s) for 'gift'.
```

- Screenshot 5 : Monthly summary report with budget warning

```
--- Menu ---
1. Add new entry
2. View all entries
3. Edit an entry
4. Delete an entry
5. Search by category
6. Sort entries
7. Monthly summary report
8. Exit
-----------------------------

Enter your choice (1-8): 7

--- Monthly Summary Report ---
Enter year (e.g. 2026) or 'back': 2026
Enter month (1-12) or 'back': 3

Year: 2026   Month: 03

  Total Income   : ৳17000.00
  Total Expense  : ৳6500.00
  Net Balance    : ৳10500.00
  Top Category   : Other
```

```
--- Menu ---
1. Add new entry
2. View all entries
3. Edit an entry
4. Delete an entry
5. Search by category
6. Sort entries
7. Monthly summary report
8. Exit
-----------------------------

Enter your choice (1-8): 7

--- Monthly Summary Report ---
Enter year (e.g. 2026) or 'back': 2026
Enter month (1-12) or 'back': 3

 Year: 2026   Month: 03

  Total Income   : ৳17000.00
  Total Expense  : ৳17500.00
  Net Balance    : ৳-500.00
  Top Category   : Rent

Warning: You have spent 80% or more of your income!
```